NTNU
Kunnskap for en bedre verden

Department of Mechanical and Industrial Engineering

TMM4935 - Industrial ICT, Master's Thesis

Development of video streaming tool for Remote Train Operations

Author:
Einar Thingstad Myhrvold

Student number:
543951

Supervisor:
Nils O.E. Olsson

Date: 18.03.2026

Preface

This master thesis

The development of ATO at NTNU has been in progress for multiple years and is now in the remote operation phase. Master students at the Department of Mechanical and Industrial Engineering has steadily worked on this project. In the fall of 2024 and spring of 2025 it was from Emilia Kozarevic, and I will continue her and NTNU's research on the topic.

Subject & knowledge listed below has been used.

- TBA4225 - Railway Engineering 2
- IT2810 - Web Development
- TDT4240 - Software Architecture
- TDT4120 - Algorithms and Data Structures

I would like to end the preface by thanking Nils Olsson for the work of supervising me during this semester and the effort he has put in to help me write this paper. I also would like to thank Patrick Urassa (NTNU) and Tomas Rosberg from VTI for discussing and the insight in the project.

Abstract

Norwegian

Table of Contents

List of Figures

List of Tables

1 Introduction

As trains become automated, the need for operators to monitor and control them remotely is growing. To do this safely, the operator needs a live video feed from the train that is stable and fast enough to react to what is happening on the track. This thesis looks at how such a video streaming system can be built, tested and evaluated, with a focus on finding configurations that work well for remote train operation.

1.1 Background and Motivation

As the railway sector transitions toward higher Grades of Automation (GoA), particularly GoA 3 and GoA 4, remote driving becomes an increasingly critical operational capability. Remote driving is not only a natural consequence of reduced onboard personnel at higher GoA levels, but serves as a mandatory safety layer within the ATO architecture enabling operators to intervene when automated systems encounter situations they cannot resolve independently, such as unexpected track conditions or degraded operational modes [source].

A fundamental requirement for safe remote driving is the availability of reliable, low-latency video streaming. When the operator is physically separated from the train, the video feed becomes the primary source of situational awareness, and its quality and responsiveness directly determine whether timely and safe intervention is possible [source]. Industry experience further demonstrates that remote operation can support a range of practical tasks including shunting, depot movements, and vehicle preparation provided that the communication link meets sufficient performance criteria [source].

Understanding the relationship between latency and human perception is equally important in this context. Research on human factors in remote operation indicates that video delay, bitrate, and stream stability all influence an operator's ability to detect obstacles, interpret dynamic scenes, and execute corrective actions under time pressure [source]. Identifying the latency boundaries within which safe remote operation remains feasible is therefore a key step toward enabling higher automation levels in railway systems.

This thesis is developed within an ongoing research collaboration at NTNU, in partnership with VTI and other European railway organizations, contributing to the technical groundwork required to make remote train driving a safe and practical reality in future railway deployments.

1.2 Problem

The current remote train operation setup relies on a commercial product, Voysys, which imposes significant limitations on the project's ability to test and develop the system further. As a closed external solution, it restricts access to the underlying pipeline, making it difficult to adjust configurations, measure internal latency, or experiment with different streaming approaches. In a research context where iterative testing and configuration control are essential, this represents a fundamental obstacle. In addition, licensing such a product over the course of a long term research project represents a considerable and ongoing cost.

At the same time, there is limited existing research that addresses video streaming specifically in the context of remote train operation. Most work on low latency video transmission is either too general or focused on other domains, leaving a gap when it comes to understanding what configurations and compromises are most suitable for railway use cases [source].

Building a custom streaming solution also comes with its own set of challenges. Network conditions such as signal strength, interference, and varying load introduce unpredictable variability in transmission quality. Choosing the right codec, transport protocol, and encoder settings requires systematic testing, as each parameter affects latency, quality, and resource usage in ways that are not always predictable. Balancing video quality against latency is a difficult comparison and testing

these differenes in settings on real hardware in a realistic setup adds further complexity. Without a flexible and open pipeline, it is difficult to isolate and measure where latency is introduced and how different configurations affect overall system performance.

1.3 Objectives

The main objective of this thesis is to design and implement a low-latency video streaming pipeline tailored specifically for remote train operation, serving as a practical replacement for the current commercial solution. The pipeline should be functional, flexible, and open enough to support continued development and testing within the project. More specifically, the objectives are to:

1. O1: Develop a working GStreamer-based streaming pipeline that meets the low-latency requirements of remote train operation
2. O2: Implement a latency measurement framework that allows end-to-end delay to be captured, logged, and compared across different configurations
3. O3: Evaluate the pipeline across multiple configurations, including different codecs, transport protocols and network settings, to identify the interplay between latency and video quality
4. O4: Design the system to be dynamic and extensible, allowing additional cameras and new configurations to be added with minimal effort
5. O5: Deliver a stable and usable prototype that is good enough for practical use during the current development period and serves as a foundation for future work

The result should be a system that gives the project full control over the streaming pipeline, removes dependency on external licensed software, and produces reliable data to guide future configuration decisions.

1.4 Research Questions

The following research questions guide the experimental work of this thesis:

- RQ1: What end-to-end latency and video quality can be achieved in a GStreamer-based video streaming pipeline for remote train operation, and which codec and transport configuration performs best?
- RQ2: How do network conditions, including transport protocol and available bandwidth, affect streaming latency and reliability in a remote train operation context?
- RQ3: What are the practical interplay between latency, video quality, and resource usage across different streaming configurations, and which configuration is most suitable for remote train operation?

1.5 Scope and Limitations

The scope of this thesis covers the design, implementation and experimental evaluation of a custom video streaming pipeline for remote train operation. The system is developed as a proof of concept and is not intended as a deployment ready solution. Development and pre testing was conducted in a controlled indoor environment, while final testing was performed from a moving vehicle to simulate realistic train movement conditions.

The experimental evaluation focuses on the parameters identified as most relevant to remote train operation based on prior research [source: prosjektoppgave], and selected based on time and hardware availability. The configurations tested include codec selection (H.264, H.265 and MJPEG),

transport protocol (UDP and TCP), encryption overhead (SRTP), bandwidth conditions, resolution, and GPU versus CPU based encoding. While GStreamer allows a large number of additional parameters to be adjusted, such as buffer sizes, queue configurations, bitrate control modes and keyframe intervals, expanding the test matrix further was not feasible within the time constraints of a single master thesis. The chosen parameters are considered sufficient to identify the most significant performance differences relevant to the use case.

Latency is measured end to end between the point where an RTP packet enters the sender pipeline and the point where the corresponding decoded frame arrives at the receiver display sink. Camera capture latency and screen rendering latency are not included in these measurements, as they occur outside the instrumented portion of the pipeline and are considered largely constant across configurations. Clock synchronization between the sender and receiver relies on internet based time sources, introducing a small margin of uncertainty estimated at a few milliseconds, which is acknowledged when interpreting absolute latency values.

The thesis does not include testing on an actual operational train. A moving vehicle was used to simulate train conditions, but real world railway infrastructure, operational speeds, and trackside network coverage are outside the scope of this work and represent a natural next step for future validation.

The thesis does not include formal safety certification or a RAMS analysis in accordance with EN 50126. While the system is developed with safety critical use in mind, evaluating compliance with railway safety standards is considered outside the scope of this work and is left for future development phases.

1.6 Outline of Structure

The remainder of this thesis is organized as follows:

Chapter 2 presents the theoretical background underlying the work. This includes an overview of Automatic Train Operation and the role of remote driving at higher GoA levels, fundamentals of video streaming including codecs, transport protocols and payloading, an introduction to GStreamer and its core concepts, the main sources of latency in a streaming pipeline, relevant aspects of cellular and WiFi communication, and the metrics used to evaluate streaming performance.

Chapter 3 describes the design of the streaming system. This covers the requirements that shaped the system, the overall architecture and component choices, the GStreamer pipeline design for both sender and receiver, the approach to timestamp based latency measurement, and the network setup used in testing.

Chapter 4 explains how the research was conducted. This includes the overall research design, the iterative development process and how it shaped the system over time, the GStreamer pipeline development process, and the experimental and latency measurement methodology used to evaluate the system.

Chapter 5 presents the experimental results. This covers a comparison of GStreamer configurations across different codecs, transport protocols and encoding approaches, latency measurements per configuration, the effect of network conditions on streaming performance, and the interplay between video quality and latency across configurations.

Chapter 6 interprets and reflects on the results in relation to the research questions and the broader context of remote train operation.

Chapter 7 summarizes the key findings and contributions of the thesis and outlines directions for future work, including testing on real railway infrastructure and further expansion of the configuration space.

2 Theory

This chapter provides the theoretical background needed to understand the work in this thesis. Beginning with Automatic Train Operation and the role of remote driving within that framework, it covers the fundamentals of video streaming including codecs and transport protocols, introduces GStreamer as the core technology used in the pipeline, describes the main sources of latency in a streaming system, and discusses the relevant characteristics of the communication technologies used in testing.

2.1 Remote Train Operations

Remote Train Operations (RTO) refers to the ability to monitor and control a train from a location outside the cab, typically a centralised control room, using live video feeds and remote control interfaces. As railways move toward higher levels of automation, RTO has become an important part of safe and practical deployment. It allows human operators to intervene when automated systems encounter situations they cannot handle on their own, and it serves as a required fallback layer for trains operating without a driver or on-board staff [1, 2].

2.1.1 Grades of Automation

The Grade of Automation (GoA) is a classification system defined in the international standard IEC 62290–1 that describes how much of the train's operation is handled automatically versus by a human [3]. The standard defines five levels, from GoA 0 to GoA 4, where each higher level shifts more responsibility from the driver to the automated system. Table 1 summarises the levels.

Table 1: Grades of Automation as defined in IEC 62290–1 [3, 4].

GoA	Name	Description
GoA 0	On-sight operation	Fully manual with no automatic protection.
GoA 1	Manual operation	Driver controls the train; Automatic Train Protection (ATP) enforces speed limits and safe separation.
GoA 2	Semi-automated (STO)	ATO handles acceleration and braking; a driver remains on board for doors, obstacle response, and degraded modes.
GoA 3	Driverless (DTO)	No driver needed in normal operation. On-board staff may be present for passenger assistance and emergencies.
GoA 4	Unattended (UTO)	Fully automatic operation with no staff on board. Remote supervision and control are required for incidents.

RTO is most relevant from GoA 3 upward. At GoA 3, on-board staff can still respond to incidents directly, but remote oversight is used for operational management. At GoA 4, where no staff are on board, RTO becomes the primary mechanism for human intervention when automated systems fail or encounter unexpected conditions [5, 6].

2.1.2 Role of Video in Remote Operation

A remote operator who cannot physically see or hear the train must rely entirely on what the system provides. In practice, this means live video from cameras mounted on or near the train is the most important source of situational awareness [7, 8]. Without a stable and clear video

feed, the operator cannot assess track conditions, detect obstacles, interpret signals, or make safe decisions about speed and braking.

Research on human factors in remote driving consistently shows that video quality has a direct effect on operator performance. Studies on bitrate, frame rate, and resolution indicate that reductions in bitrate lead to measurable drops in both reaction speed and accuracy, while changes in frame rate have a smaller effect within typical operating ranges [7]. This suggests that encoding and transmission choices in the video pipeline are not just technical decisions but directly influence operational safety.

Beyond individual video quality, remote train driving typically requires multiple simultaneous camera views. A camera facing straight forward is needed for obstacle and signal detection, while side cameras support the operators experience, needed for immersivenes of speed and movement like you can see a Figure 1 [1, 2, 9].

Figure 1: Three-screen teleoperation simulator setup used by Neumeier et al. to evaluate the effect of latency on remote driving performance [9].

2.1.3 Latency Requirements and Regulations

Latency is a central performance parameter in a RTO video system. If the delay between what happens on the track and what the operator sees on screen is too large, safe intervention becomes impossible. As of now, a big challenge is that no single universally agreed threshold exists for remote train operation, and most values currently used in the field are borrowed from research on cars, drones, and cranes [10].

The most relevant regulatory reference comes from 3GPP TS 22.289, a technical specification developed in cooperation with the International Union of Railways (UIC) and the European Telecommunications Standards Institute (ETSI) [11]. It classifies remote video control at GoA 3 and GoA 4 as *Very Critical Video Communication* and sets a target end-to-end latency of ≤ 100 ms at speeds up to 500 km/h, or ≤ 10 ms at speeds up to 40 km/h, with a reliability requirement of 99.9% and a data rate of 10–20 Mbps. These values are defined at the communication interface and are not yet formally adopted into ERA's Technical Specifications for Interoperability (TSIs) [11].

Empirical studies from railway demonstrations report glass-to-glass (G2G) latency values of 340–380 ms for tramway remote driving using commercial systems [12], while the Kozarevic study at NTNU measured an average of around 212 ms using a GPS-synchronised measurement method [8]. Human factors research from other vehicle domains suggests that operators begin to adjust their behaviour at delays above 250–300 ms, and that latency variability can also be more disruptive than a consistently higher but stable delay [9, 10].

Together, these findings motivate the work in this thesis: to build and evaluate a custom streaming pipeline where each component of the latency budget can be measured, understood, and optimised, rather than relying on a black-box commercial system.

2.2 Clock Synchronisation

Accurate latency measurement between two separate machines requires that both share a common time reference. Each computer maintains an internal clock driven by a local hardware oscillator. Even at identical nominal frequencies, two independent oscillators will accumulate a growing offset over time due to differences in temperature, aging, and manufacturing tolerances [13]. When a timestamp recorded on one machine is compared against a timestamp recorded on another, any offset between the two clocks appears directly in the measured result. Depending on the magnitude of the drift, this error can easily exceed the latency values being measured, making clock synchronisation a requirement for accurate one-way delay measurement.

2.2.1 GPS-Based Clock Synchronisation

The Global Positioning System (GPS) operates satellites that each carry onboard atomic clocks traceable to Coordinated Universal Time (UTC). Each satellite broadcasts a navigation message containing the current time along with parameters allowing a receiver to compute the signal propagation delay from the satellite to the receiver. By combining signals from at least four satellites and solving for the receiver position and clock offset simultaneously, a GPS receiver can derive a precise absolute time reference [14].

A common timing output from GPS receivers is the pulse-per-second (PPS) signal. This is a periodic electrical pulse whose rising edge is aligned to the UTC second boundary. Commercial GPS receivers designed for timing applications typically achieve a PPS accuracy of 100 nanoseconds or better relative to UTC, and precision timing receivers can reach the range of tens of nanoseconds under stable conditions [13, 15]. The PPS signal provides a hardware-level time reference that is independent of network conditions and available continuously as long as the receiver has a clear view of the sky.

When a computer's operating system is disciplined by a GPS PPS signal, the kernel timestamps each incoming pulse and uses the known UTC alignment to adjust the phase and frequency of the system clock continuously. The resulting clock closely tracks UTC, with residual errors typically in the sub-microsecond range [15]. For latency measurements in a video streaming system where delays are on the order of tens to hundreds of milliseconds, this accuracy level means that clock error contributes negligibly to the measured values. GPS-based synchronisation is therefore used in studies where high measurement precision is required and where both endpoints can be equipped with a GPS receiver, as seen in the latency measurement work of Kaknjo et al. [16] and the tramway remote driving tests reported in FP2R2DATO D41.2 [12].

2.2.2 Network-Based Clock Synchronisation with Chrony

When a hardware GPS reference is not available, clock synchronisation can be achieved over a network using the Network Time Protocol (NTP). NTP operates by exchanging timestamped packets between a client and one or more time servers. The client records the time it sent the request, the time the server received it, the time the server sent its reply, and the time the client received the reply. From these four timestamps, NTP estimates the round-trip propagation delay and uses half of that value to approximate the one-way delay, computing a correction to the local clock [17].

Chrony is an NTP implementation for Linux systems that is designed to maintain accurate synchronisation under variable network conditions. Rather than making abrupt step corrections to the clock, Chrony adjusts the frequency of the local oscillator continuously, which reduces disruption to running processes and improves long-term stability [18]. It also tracks the estimated error of each configured time source and selects the most reliable one based on a combination of stratum, distance, and measured jitter.

When synchronised to public internet time servers such as those provided by the `pool.ntp.org` project, Chrony typically achieves a clock accuracy within a few milliseconds over an internet

connection. On a local area network under stable conditions the accuracy improves to tens of microseconds, and with hardware timestamping enabled on the network interface, where the network card rather than the kernel records packet arrival and departure times, sub-microsecond accuracy is possible [19, 20].

2.3 Video Streaming

Video streaming is the continuous transmission of video data from a source to a receiver over a network, allowing the receiver to display the content in real time without storing the full recording first [21]. In a remote train operation context, this means live footage from cameras on or near the train is captured, compressed, sent across a network, and displayed on the operator's screen with as little delay as possible.

A video streaming system can be thought of as a chain of components, each with its own role. At the sending end, raw video is captured by a camera and handed to an encoder, which compresses it into a format suitable for transmission. The compressed data is then packaged into a transport protocol and sent across the network. At the receiving end, the process is reversed: packets are reassembled, decoded back into raw frames, and rendered on screen. The choices made at each step, which codec, which protocol, which network directly affect the end-to-end latency and video quality that the remote operator experiences [10, 22].

2.4 Video Compression

Video compression works by removing redundancy from the raw video signal. There are two main redundancy that codecs use for compression. Spatial redundancy exists within a single frame where neighbouring pixels often have similar values. Temporal redundancy exists across frames where much of the image remains unchanged between one frame and the next. A video encoder exploits both types of redundancy to represent the video with fewer bits than the original raw data. Compression can be lossless where the original data can be perfectly reconstructed, or lossy where some information is discarded in exchange for a much smaller file size. Most video streaming applications use lossy compression, as lossless video requires far more bandwidth than most networks can provide [23, 24].

2.4.1 H.264

H.264, also known as Advanced Video Coding (AVC), is a video compression standard developed jointly by the ITU-T Video Coding Experts Group and the ISO/IEC Moving Picture Experts Group [25]. It has become the dominant codec for real-time video applications including video conferencing, IP surveillance, and live streaming, due to its balance between compression efficiency, computational cost, and broad hardware support [26].

The codec reduces redundancy across two dimensions. Within a single frame, intra-frame prediction estimates pixel values from neighbouring blocks and stores only the difference. Across consecutive frames, inter-frame prediction identifies how blocks have moved and stores motion vectors rather than full pixel data, allowing H.264 to represent video at significantly lower bitrates than older standards such as MPEG-2 while maintaining comparable visual quality [25].

An important concept is the Group of Pictures (GOP), which defines how often a full intra-coded frame appears in the stream. Short GOP intervals are preferred for real-time applications to reduce both start-up delay and recovery time after packet loss, at the cost of a slightly higher average bitrate [26]. How the encoded data is structured for network transport is described in Section 2.4.4.

2.4.2 H.265

H.265, also known as High Efficiency Video Coding (HEVC) and formally standardised as ITU-T H.265 and ISO/IEC 23008–2, is the successor to H.264 published in 2013 [27]. Its primary goal was to achieve roughly 50% better compression compared to H.264 at equivalent visual quality [28].

HEVC achieves its efficiency gains by replacing the fixed 16×16 macroblock structure of H.264 with a flexible Coding Tree Unit (CTU) architecture that supports block sizes up to 64×64 pixels. Larger blocks handle uniform image regions efficiently, while smaller blocks are used in areas with fine detail or motion. This adaptive structure allows the encoder to allocate coding effort where it is most needed, reducing the total number of bits required to represent a frame at a given quality level [27].

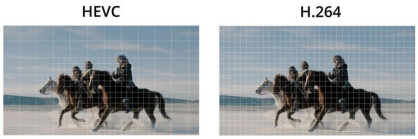

Figure 2: [23].

The main difference compared to H.264 is computational cost. Encoding with H.265 requires significantly more processing power, which increases encoding time and places greater demand on the sender hardware [28].

2.4.3 Frame Types

Both H.264 and H.265 organise their compressed video into three frame types, each with a different role in the compression scheme [29].

An I-frame, or intra-coded frame, is compressed using only the data within the frame itself, with no reference to any other frame. It can therefore be decoded independently and serves as a recovery point in the stream. A P-frame, or predictive frame, stores only the differences relative to a previously decoded I or P frame, reducing the amount of data needed to represent a frame where little has changed. A B-frame, or bidirectionally predicted frame, can reference both a preceding and a following frame, achieving even higher compression by interpolating between two reference points [29].

The sequence of frame types between two consecutive I-frames is called a Group of Pictures (GOP). A shorter GOP means I-frames appear more frequently, which increases the average bitrate but improves the ability to recover from packet loss and reduces the delay before a new receiver can begin decoding the stream.

2.4.4 NAL Units

Both H.264 and H.265 organise their encoded bitstream through a Network Abstraction Layer (NAL). The NAL divides the encoded data into discrete units called NAL units, each consisting of a header that identifies the type of data it contains, followed by the payload [25]. There are two categories: Video Coding Layer (VCL) NAL units carry the compressed image data, while non-VCL NAL units carry decoder configuration information, most importantly the Sequence Parameter Set (SPS) and Picture Parameter Set (PPS). The SPS contains configuration that applies to the entire video sequence such as resolution and profile, while the PPS contains parameters applying to individual pictures such as entropy coding mode. A decoder must receive both before it can decode any VCL data [30].

When H.264 or H.265 is carried over RTP, each NAL unit is mapped to one or more RTP packets. If a NAL unit exceeds the network maximum transmission unit (MTU), it is fragmented across

multiple consecutive RTP packets. The RTP marker bit is set on the last packet of a complete frame, signalling to the receiver that all data for that frame has arrived [31].

2.4.5 MJPEG

Motion JPEG (MJPEG) is a video format in which each frame is compressed independently as a JPEG image, with no reference to any other frame in the sequence [32]. This purely intraframe approach contrasts with H.264 and H.265, which both rely on temporal prediction across frames to achieve compression.

Because each frame carries all the data needed for its own decoding, MJPEG has two notable properties relevant to remote operation. First, packet loss affects only the frame it occurs in, with no propagation of errors to subsequent frames. Second, encoding and decoding complexity is low and predictable, as the encoder processes each frame in isolation without any lookahead or motion estimation [33]. These properties can result in lower and more consistent encoding latency compared to inter-frame codecs under the same hardware conditions.

The main limitation of MJPEG is its high bandwidth consumption. Without temporal compression, every frame must carry its full spatial data, which results in bitrates several times higher than H.264 at equivalent quality [34].

2.5 Transport Protocols

2.5.1 RTP and SRTP

The Real-time Transport Protocol (RTP) is a network protocol designed for the delivery of audio and video over IP networks, standardised by the IETF in RFC 3550 [35]. It operates at the application layer and is typically carried over UDP, adding a structured header on top of each media payload. The header contains a sequence number that increments with each packet, allowing the receiver to detect packet loss and reorder out of order arrivals. A timestamp reflects the sampling time of the media content and is used to pace playback at the correct rate on the receiving end. The marker bit signals significant events in the stream, most commonly the last packet belonging to a video frame, so the receiver knows when a complete frame has arrived and is ready for decoding [35].

RTP is typically paired with the RTP Control Protocol (RTCP), which runs on a separate port alongside the media stream. RTCP carries periodic reports containing statistics such as packet loss, jitter, and round trip delay, giving both sides visibility into network conditions without carrying any media itself [35].

The Secure Real-time Transport Protocol (SRTP) is a security profile built on top of RTP, defined in RFC 3711 [36]. It adds AES encryption to protect the payload, HMAC authentication to verify packet integrity, and replay protection to reject duplicate packets. Each packet is encrypted independently, so the loss of one packet does not affect the decryption of those that follow. The added overhead per packet is small and the computational cost on modern hardware is low [36].

2.5.2 UDP

The User Datagram Protocol (UDP) is a transport layer protocol defined in RFC 768 [37]. It provides a minimal communication mechanism for sending packets, called datagrams, between applications over an IP network. UDP does not establish a connection before sending, makes no guarantee of delivery or packet ordering, and does not retransmit lost packets. Each datagram is handled independently, and the protocol adds only an eight byte header containing source port, destination port, length, and a checksum for basic integrity verification [37].

The absence of connection setup and retransmission mechanisms means UDP introduces very little overhead and no additional delay beyond the time required to transmit the data itself. This makes

it well suited for applications where low latency is more important than guaranteed delivery, such as real-time audio and video streaming, where a retransmitted packet arriving late is less useful than a fresh one [38].

2.5.3 TCP

The Transmission Control Protocol (TCP) is a transport layer protocol defined in RFC 793 [39]. Unlike UDP, TCP is connection oriented, meaning that a logical connection is established between two endpoints through a handshake process before any data is exchanged. TCP guarantees reliable, ordered delivery by assigning sequence numbers to all transmitted bytes and requiring the receiver to send acknowledgements back to the sender. Any packet that is not acknowledged within a timeout period is retransmitted automatically [39].

TCP also implements flow control and congestion control mechanisms. Flow control prevents the sender from transmitting data faster than the receiver can process it, using a sliding window that the receiver advertises. Congestion control reduces the transmission rate when packet loss is detected, interpreting loss as a signal of network congestion [38]. These mechanisms make TCP well suited for applications that require complete and correct data delivery, such as file transfer and web browsing, but the added overhead means the transmission rate can vary depending on network conditions [38].

2.6 Streaming Parameters

2.6.1 Bandwidth

Bandwidth in video streaming refers to the amount of data that can be transmitted per unit of time across a network connection, typically measured in megabits per second (Mbps). In a streaming pipeline the term bitrate is often used alongside bandwidth: bitrate describes how much data the encoder produces per second, while bandwidth describes how much the network can carry. When bitrate exceeds available bandwidth, packets accumulate in buffers or are dropped, leading to increased latency, visual degradation, or stream interruptions [21].

The required bandwidth depends on the codec, resolution, frame rate, and scene complexity. A higher bitrate generally produces better visual quality, but the relationship is not linear. Subjective testing has shown that perceived quality improves sharply at low bitrates and then levels off as bitrate increases, meaning that doubling the bitrate does not double the perceived quality [40].

2.6.2 Resolution

Resolution defines the number of pixels in each video frame, expressed as width by height. Common values include 1280×720 (720p) and 1920×1080 (1080p). A higher resolution captures more spatial detail, but also increases the amount of data per frame that the encoder must compress and the network must carry [40].

For a given codec and bitrate, increasing resolution without increasing the bitrate results in more compression per pixel, which can reduce sharpness and introduce visible artefacts. Conversely, a high resolution with a sufficiently high bitrate produces a clearer image. The 3GPP Rel-16 specification for video based remote railway control recommends a minimum of 1920×1080 at 30 frames per second as the target output quality [22].

2.7 Latency Sources in a Video Streaming Pipeline

The total delay from when a video frame is captured by a camera to when it is displayed on the operator's screen is commonly referred to as glass-to-glass latency. This delay does not originate

from a single component but accumulates across each stage of the streaming pipeline [41]. Understanding where latency is introduced at each stage is necessary for identifying bottlenecks and evaluating the overall performance of a streaming system.

The pipeline can be divided into five sequential stages: capturing, encoding, transmitting, decoding, and rendering. Each stage contributes a portion of the total delay, and the contribution of each depends on the hardware, configuration, and network conditions in use [16]. The following subsections describe each stage and the mechanisms through which latency is introduced.

2.7.1 Capturing

Capturing latency is the delay introduced at the camera before any data enters the network pipeline. It originates from two main sources: the exposure time of the image sensor and the subsequent image processing performed inside the camera [42].

The image sensor captures light across all its pixels during one exposure interval before converting the accumulated light energy into digital signals. At 30 frames per second the sensor collects light for approximately 33 milliseconds per frame, meaning that any event occurring just after the start of an exposure interval must wait until the end of that interval before it is registered [42]. Higher frame rates reduce this waiting time proportionally.

After exposure, the raw sensor data passes through a series of internal processing steps such as demosaicing, noise reduction, and scaling before being handed to the encoder. The time required for these steps depends on the camera hardware and the features enabled, and can add several milliseconds to the total capture delay [16, 42]. In network cameras the combined capture and internal processing time is typically under 50 milliseconds, though it varies between camera models and frame types.

2.7.2 Encoding

Encoding latency is the delay introduced while the encoder compresses raw video frames into a transmittable format. It originates from the computational steps required to apply intra and inter frame prediction, transform coding, and entropy coding to each frame [42].

Buffering is another significant source of encoding delay. Encoders that use B-frames must hold one or more future frames in memory before the current frame can be encoded, since B-frames reference both past and future frames. This lookahead buffering adds at least one frame period of delay before any data can be output [43]. Encoders configured for low latency operation typically disable B-frames and lookahead to eliminate this waiting time, at the cost of some compression efficiency.

The computational load of the encoding process itself also contributes to delay. More complex encoding settings produce better compression but require more processing time per frame. When the encoding time for a single frame exceeds the frame period, frames begin to queue and latency accumulates over time. The choice of encoder preset and hardware therefore directly affects how much delay the encoding stage adds to the pipeline [42].

2.7.3 Transmitting

Transmission latency is the delay introduced while encoded video data travels across the network from the sender to the receiver. It consists of several components that accumulate along the path [38].

Propagation delay is the time required for a signal to travel physically from the sender to the receiver, determined by the distance between the two endpoints and the speed of signal propagation through the medium. Serialisation delay is the time required to place all the bits of a packet onto the transmission link, determined by packet size and link capacity. Queuing delay occurs when

packets wait in buffers at intermediate nodes, such as routers and switches, before being forwarded. In congested networks this can vary considerably from packet to packet [38].

Variation in the arrival time of packets at the receiver is called jitter. When packets that were sent at regular intervals arrive with uneven spacing, the receiver cannot immediately display them in order. To compensate, a jitter buffer is used on the receiver side to hold incoming packets briefly and release them at a steady rate. While this smooths out the stream, the jitter buffer itself introduces additional fixed delay. A larger buffer tolerates more jitter but increases the total latency, while a smaller buffer reduces delay at the cost of being more sensitive to network variation [42].

2.7.4 Decoding

Decoding latency is the delay introduced while the receiver decompresses incoming NAL units back into raw video frames for display. The computational steps involved are the inverse of encoding, including entropy decoding, inverse transform, and motion compensation, and their cost depends on the codec and the hardware available [43].

Frame reordering is another notable source of decoding delay. When B-frames are present in the stream, the decoder receives frames in a different order than they are to be displayed, since B-frames reference future frames that have not yet been decoded. The decoder must therefore buffer frames until their reference frames arrive before it can output them in the correct display order. The number of frames that must be held in this decoded picture buffer (DPB) depends on the GOP structure used by the encoder [43].

When the encoder is configured without B-frames, as is common in low latency streaming, the reordering delay is eliminated and the decoder can output each frame as soon as it is decoded. The remaining decoding latency is then primarily determined by the processing time per frame, which on modern hardware is typically in the range of a few milliseconds [42].

2.7.5 Rendering

Rendering latency is the delay introduced at the final stage of the pipeline, where a decoded frame is passed to the display and made visible to the viewer. It consists of two main components: the time required for the rendering application to prepare and present the frame to the graphics system, and the time the display itself takes to show it [42].

A display can only update its image at the rate defined by its refresh rate, measured in hertz. A monitor running at 60 Hz updates 60 times per second, meaning a new image appears at most once every 16.7 milliseconds. If a decoded frame arrives just after a refresh has occurred, it must wait until the next refresh cycle before it becomes visible, introducing up to one full frame period of additional latency [44]. Higher refresh rates reduce this waiting time, as the interval between refresh cycles is shorter.

The rendering application may also hold frames in a buffer before handing them to the display, either to synchronise with the refresh cycle or to ensure smooth playback. This buffering adds a predictable but fixed amount of delay. Combined with the display's own internal processing time, rendering latency is typically in the range of one to two frame periods and represents the last contribution to the total glass-to-glass delay [42].

2.8 GStreamer

GStreamer is an open source multimedia framework written in C that allows developers to construct pipelines of media processing components [45]. It was originally developed at the Oregon Graduate Institute in 1999 and has since become a widely used foundation for applications ranging from media playback and transcoding to real-time video streaming and computer vision [45]. The framework

is built around a modular, plugin based architecture where each processing step is provided by a plugin, and individual components can be combined freely to form a complete processing chain suited to the application at hand.

The following subsections describe the core concepts of GStreamer that are relevant to understanding how a streaming pipeline is constructed and instrumented.

2.8.1 Pipeline and Elements

An element is the fundamental building block in GStreamer. Each element performs a single, specific task in the media processing chain, such as reading data from a network socket, decoding a compressed video stream, converting a pixel format, or writing frames to a display. Elements are linked together in sequence to form a pipeline, where the output of one element flows into the input of the next [46].

A pipeline is a specialised container that manages a group of connected elements and coordinates their execution. It provides a shared clock that all elements use to synchronise data, and it handles state transitions for the entire graph as a unit. The three main operational states are NULL, PAUSED, and PLAYING.In the PLAYING state, data flows continuously from source elements through any intermediate processing elements and finally into sink elements [46].

Elements are broadly categorised by their role in the pipeline. Source elements produce data and have no input, for example by reading from a camera or a network port. Filter elements receive data, process it, and pass it on, covering tasks such as encoding, decoding, and format conversion. Sink elements consume data at the end of the pipeline and have no output, typically by rendering video to a screen or writing data to a socket [46]. By combining elements of these three types, a complete media processing pipeline can be assembled from reusable components without writing the processing logic from scratch.

2.8.2 Pads

Pads are the connection points through which data flows between elements in a GStreamer pipeline. Every element exposes one or more pads, and data moves from the source pad of one element to the sink pad of the next. Source pads produce data and sink pads consume it, where the terms are defined from the perspective of the element itself [47].

Each pad carries capability information, called caps, which describes the type and format of data it can handle. This includes properties such as media type, codec, resolution, and frame rate. Before two pads can be linked, GStreamer negotiates a common format by finding the intersection of their capabilities. If no common format exists the link is rejected, preventing incompatible elements from being connected [47].

Pads can be either static or dynamic. Static pads are always present on the element from the moment it is created. Dynamic pads appear only when the element begins processing data and has determined what format it is working with, which is common in demuxers and parsers that must read the stream before knowing its structure [47].

2.8.3 Sinks

A sink element is the terminal point of a GStreamer pipeline. It receives data from upstream elements and delivers it to an output destination, such as a display, a file, or a network socket. Sink elements have one or more sink pads for receiving data but no source pads, as they do not pass data further down the pipeline [48].

Sink elements are treated specially by the GStreamer core in relation to synchronisation. When a pipeline transitions to the PAUSED state, the sink waits to receive the first buffer or event before

confirming the state change. This behaviour allows the pipeline to ensure that data is available and properly configured before playback begins [48].

A common type of sink in video pipelines is a display sink, which renders decoded frames to a screen. GStreamer provides several display sink implementations suited to different platforms and windowing systems. The `sync` property controls whether the sink waits for the correct presentation time before displaying each frame. Setting `sync=false` disables this waiting and causes the sink to display frames as soon as they arrive, which reduces latency at the cost of ignoring the stream's timing information [48].

2.8.4 Probes

Probes are callbacks that an application can attach to a pad in order to monitor or intercept the data flowing through it at runtime. They allow the application to observe buffers, events, and queries as they pass through a specific point in the pipeline without modifying the pipeline structure itself [49].

A probe is installed on a pad using the `gst_pad_add_probe` function, which accepts a probe type mask and a callback function. The type mask specifies what the probe should respond to, such as the arrival of a buffer, a buffer list, or an event. Each time data matching the type mask passes through the pad, GStreamer calls the registered callback function and passes the data to it. The callback can inspect the data, read timestamps, extract header information, and return a value indicating whether the data should continue flowing, be dropped, or cause the pad to block [49].

Probes are particularly useful for latency measurement, as they allow the application to record a timestamp at one pad when data enters the pipeline and another timestamp at a downstream pad when the processed data exits. The difference between the two timestamps represents the processing time between those two points in the pipeline [50].

2.9 Network and Communication

The network connecting between the sender and receiver is a important part of any remote operation system. The choice of network technology affects how much data can be sent, how quickly it arrives, and how stable the connection is under varying conditions. The following subsections cover the network technologies relevant to this thesis.

2.9.1 Wi-Fi

Wi-Fi is a family of wireless networking technologies based on the IEEE 802.11 standard. It allows devices to communicate over a local area network using radio frequencies, most commonly 2.4 GHz and 5 GHz [51]. The 2.4 GHz band covers longer distances but offers lower maximum data rates, while the 5 GHz band supports higher data rates over shorter distances.

Wi-Fi networks operate on unlicensed radio channels where all nearby devices transmit on the same frequency. When multiple devices transmit at the same time, collisions occur and data must be retransmitted. This can increase latency and reduce predictability, particularly when the network is busy [52]. For controlled environments with few devices, Wi-Fi can deliver low and stable latency.

2.9.2 Cellular and 5G

Cellular networks provide wireless communication over wide areas by dividing coverage into cells, each served by a base station. Data travels from a device through the radio access network to a core network and then on to its destination. Each of these steps adds some delay, and the total latency depends on the network generation, radio conditions, and distance to the core [53].

The fifth generation (5G) of cellular networks was designed to support three broad service categories. Enhanced Mobile Broadband (eMBB) targets high data rates for applications such as video streaming. Massive Machine Type Communications (mMTC) connects large numbers of low power devices. Ultra Reliable Low Latency Communications (URLLC) is the category most relevant to remote control, aiming for end-to-end latency below 1 ms and a reliability of 99.999% [53, 54]. To achieve this, 5G NR introduced a shorter transmission time interval compared to 4G LTE, allowing the network to send and acknowledge data more frequently.

In practice, the latency experienced by an application over a 5G network is higher than the theoretical URLLC targets, as it also includes processing in the radio hardware, the core network, and the application itself. Studies on remote vehicle control show that total end-to-end video latency over 5G is typically in the range of tens to a few hundred milliseconds depending on the system architecture, with 5G consistently outperforming 4G in both average latency and stability [55].

2.9.3 Tailscale

Tailscale is a networking tool that creates a private mesh network between devices over the internet. It is built on top of WireGuard, an open source VPN protocol that uses modern cryptography to establish encrypted tunnels between devices [56]. Each device on the network gets a stable private IP address and can communicate directly with any other device on the same network, regardless of where they are physically located or what network they are connected to.

The main idea behind Tailscale is that devices connect directly to each other rather than routing all traffic through a central server. A coordination server handles authentication and helps devices find each other, but once a connection is established the data flows directly between the two endpoints [56]. If a direct connection is not possible due to network restrictions, Tailscale falls back to relay servers to keep traffic flowing.

WireGuard, which handles the actual data transfer, adds encryption and authentication to every packet passing through the tunnel. The protocol is designed to have low overhead compared to older VPN technologies, which keeps the additional latency introduced by the encrypted tunnel small [57].

2.10 Performance Metrics

Evaluating a video streaming system requires measuring several aspects of its behaviour under different conditions. The metrics covered in this section are latency, packet loss, throughput, CPU usage, and visual quality. Together they give a picture of how the system performs from both a network and a visual standpoint.

2.10.1 Latency

Latency in a video streaming system is the time it takes for data to travel from one point to another. The most complete measure is glass-to-glass latency, which covers the full path from when a camera captures an image to when it appears on the operator's screen [16]. However, glass-to-glass latency alone does not reveal where in the pipeline the delay originates. For this reason it is common to break the total latency into smaller segments that can each be measured independently.

Three segments are useful for understanding a streaming pipeline. Sender pipeline latency covers the time from when raw video enters the sender until encoded packets leave it. Transit latency covers the time packets spend travelling across the network. Receiver pipeline latency covers the time from when packets arrive until the decoded frame is ready for display [16].

There are several ways to measure each segment, and the choice of method has a large impact on the results. As discussed in [10], the literature uses a wide range of approaches including

round-trip time, one-way delay, and glass-to-glass timing. Round-trip time does not require clock synchronisation but measures both directions combined and cannot separate sender and receiver contributions [35]. One-way delay gives a more precise picture of a single direction but requires both endpoints to share a common time reference, as any clock offset between the two machines will appear as measurement error [16]. NTP can synchronise clocks to within a few milliseconds, while PTP achieves sub-millisecond accuracy with the right network hardware [35]. For transit latency, RTP sequence numbers provide a practical basis for measurement, as sender and receiver logs can be compared after a test to reconstruct the timing of individual packets [35].

2.10.2 Packet Loss

Packet loss occurs when one or more packets sent across a network fail to reach the receiver. In UDP based video streaming, lost packets are not retransmitted, so any data they carried is gone. For video encoded with inter-frame prediction, a lost packet can corrupt not only the frame it belongs to but also subsequent frames that reference it, until the next I-frame arrives and allows the decoder to recover [58].

Packet loss is most commonly expressed as a percentage of the total packets sent. RTP provides a direct way to detect loss, since the sequence number in each RTP packet increments by one for every packet sent. A receiver can therefore identify a lost packet whenever it observes a gap in the sequence of received numbers [35]. The packet loss rate can then be calculated by comparing the number of expected packets, derived from the sequence number range, with the number actually received.

Even small amounts of packet loss can have a noticeable effect on video quality. Research on H.264 and H.265 video streams shows that loss rates as low as 0.1% can produce visible artefacts, and that the effect becomes more severe at higher loss rates and higher resolutions [59].

2.10.3 Throughput

Throughput is the amount of data successfully transmitted from one point to another over a given period of time, measured in bits per second [38]. It describes the actual data transfer rate observed during operation, as distinct from bandwidth, which represents the maximum capacity of the link. Throughput is always equal to or lower than bandwidth, as it is affected by overhead, congestion, and packet loss [38].

In a video streaming context, throughput reflects how much encoded video data is leaving the sender and reaching the network per second. Monitoring throughput over time shows whether the sender is transmitting at a consistent rate and whether the output matches the target bitrate set by the encoder. Variations in throughput can indicate congestion on the network or changes in the encoder output caused by scene complexity [21].

2.10.4 CPU Usage

CPU usage is the percentage of time a processor spends actively running instructions during a given measurement interval, as opposed to sitting idle [60]. It is typically reported as an average across all cores or per individual core, and is measured continuously to capture how load varies over time.

In a video streaming pipeline, the most CPU intensive task on the sender side is encoding. The computational load depends on the codec, the encoder settings, the resolution, and the number of streams being processed simultaneously. When CPU usage remains consistently high, the encoder may not be able to process frames fast enough to keep up with the incoming video rate, which can cause frames to queue and latency to increase [24].

CPU usage is commonly measured using system monitoring tools that read data from the operating system at regular intervals. On Linux systems, the `sar` utility from the `sysstat` package is a

standard tool for this purpose. It samples CPU activity at a set interval and reports metrics such as the percentage of time spent in user space, kernel space, and idle [60].

2.10.5 Visual Quality

Visual quality in a video streaming context refers to how well the content received at the display represents what was originally captured. Compression, packet loss, and network conditions can all introduce artefacts such as blocking, blurring, or pixelation that reduce the clarity of the image [59].

Image quality assessment (IQA) methods are used to measure these effects in a consistent and repeatable way. They fall into two broad categories. Full reference methods compare a received image against an original, undistorted reference image. Common examples are Peak Signal to Noise Ratio (PSNR) and the Structural Similarity Index (SSIM), both of which quantify the difference between the two images using pixel level calculations [61]. These methods require access to the original image, which is not always available in a live streaming scenario.

No reference methods, also called blind quality assessment, estimate quality from the received image alone without any original to compare against [62]. One well established no reference metric is the Blind/Referenceless Image Spatial Quality Evaluator (BRISQUE), proposed by Mittal et al. [62]. BRISQUE analyses the statistical properties of the image using a model of how natural, undistorted images are expected to look. When an image contains distortions such as blur, noise, or compression artefacts, its statistical properties deviate from those of a natural image. BRISQUE quantifies this deviation and outputs a score between 0 and 100, where lower scores indicate better quality and higher scores indicate more visible distortion. The model was trained on a dataset of human quality ratings and is designed to correlate with how a human observer would rate the image [62].

3 System Design

This chapter describes the design of the video streaming system developed in this thesis. It outlines the requirements that shaped the system, the overall architecture, the pipeline design, the approach used for timestamp-based latency measurement, and the Wi-Fi network setup used during testing. Together these sections document the reasoning behind the key design decisions made throughout the development process.

3.1 Requirements

The requirements for the system are shaped by the operational context of remote train operation, where reliability and low latency are critical. Unlike a controlled lab setting, a moving train introduces real-world challenges such as varying network conditions, physical vibrations, and the need for the system to perform consistently during operation.

The key requirements identified for the system are:

Low latency: The pipeline must deliver video with as little end-to-end delay as possible, as latency directly affects the remote operator's ability to perceive and react to the environment safely.

Reliability: The system must maintain a stable stream under varying network conditions, as interruptions or degraded quality during operation could have serious safety implications.

Multi-camera support: The system must be capable of handling multiple camera streams simultaneously, reflecting the practical needs of a remote driving setup where several viewing angles are required.

Deployable on a moving train: The system must function in a real train environment, accounting for the network and physical conditions that come with that setting.

Reproducibility: The setup must be straightforward to reproduce and redeploy, both for continued development within the project and for potential use by other researchers or partners in the collaboration.

3.2 Architecture

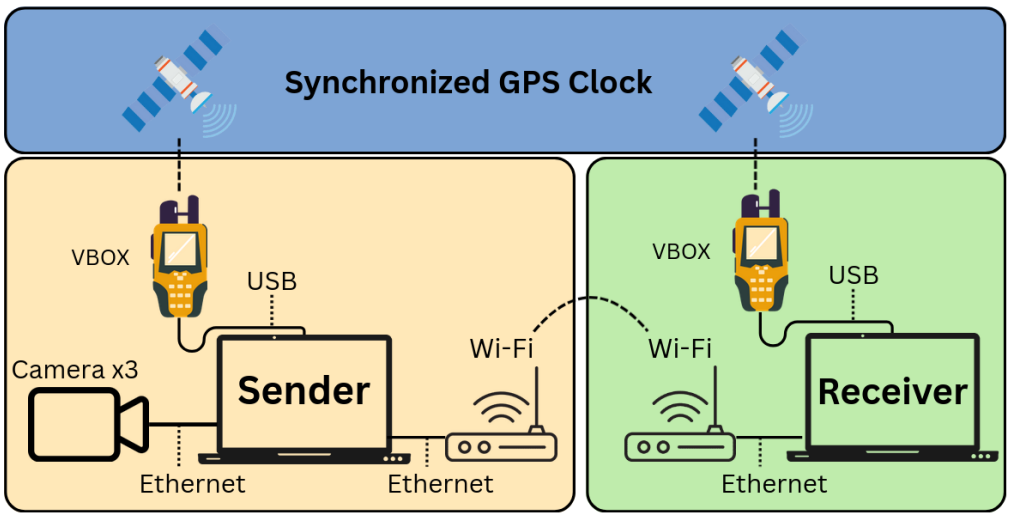

Figure 3: Physical Setup of Project

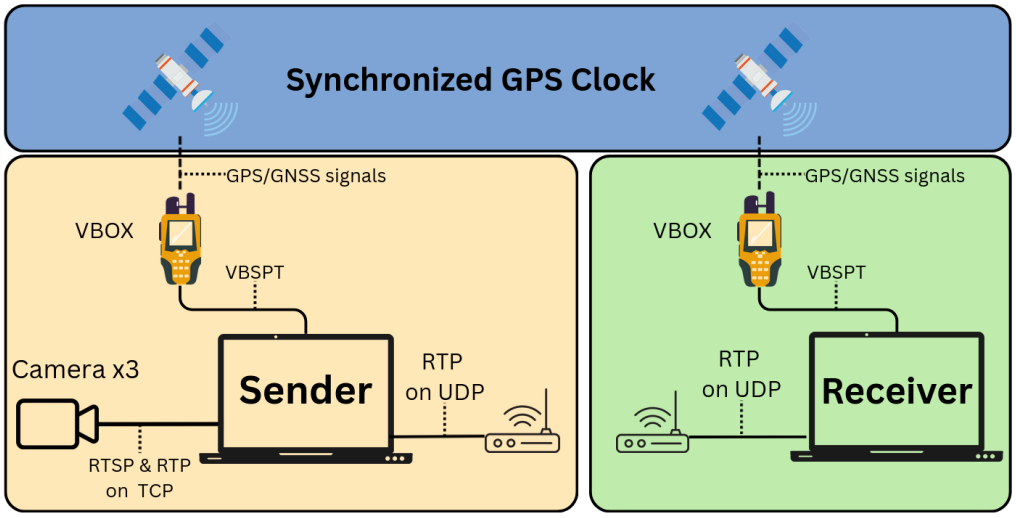

Figure 4: Application and Transport layer Setup of Project

Table 2: Setup for IP and Port Connection

Camera	Camera IP	Connection interface	Local IP	Receiver IP	RTP Port
Camera 1	192.168.0.100	eth0	192.168.0.50	10.238.111.249	5000
Camera 2	192.168.1.101	eth1	192.168.1.50	10.238.111.249	5002
Camera 3	192.168.2.102	enp0s31f6	192.168.2.50	10.238.111.249	5004

IP Cameras: Logged in with SADE, iVMS and Hikvision, to be set to standard IP address of 192.168.1.100, 101 and 102. This will remain IP address until manually overwritten.

Important to separate: Transport Layer: TCP, UDP Application Layer: RTSP, RTP TCP perfect, correct order and intact. UDP speedier but less reliable

Between Cam and Laptop, TCP. A NetCat TCP handshake to port 554 to check its availability.

Sender Computer Sets local IP address to 192.168.1.20 to make sure to be on the same subnet as the IP cameras. Pings local IP address of cameras connected to confirm possible connection. Checks RTSP port is available by using NetCat. NetCat sends a TCP handshake. RTSP message over TCP, asking for stream. Describe Stream, Camera describe, Setup up stream, Camera ready, Play stream. Setup because we ask for protocol TCP, we go Interleaved: RTSP and 2 Channels inside TCP connection. Channel 0 carries RTP (video data) Channel 1 carries RTCP (Control and monitoring QoS, Sync streams, provides feedback)

Real-Time Streaming Protocol control the stream. Using NetCat (needs download) to check if the RTSP port is open

Pulls the stream from the camera over RTSP/TCP, with no jitter buffer “latency = 0“

Camera sends: ——————————— (perfectly even) Network delivers: —— —— ——— ——-——— (uneven gaps)

Without buffer: —— ———— ——-——— (choppy, uneven) With buffer: waits...then ——————————— (smooth, even)

Because of local wired connection, the jitter is probably very low, and not adding jitter buffer will not affect much other than lowering latency.

rtph264depay, Strips the stream of RTP giving raw H.264 NAL units

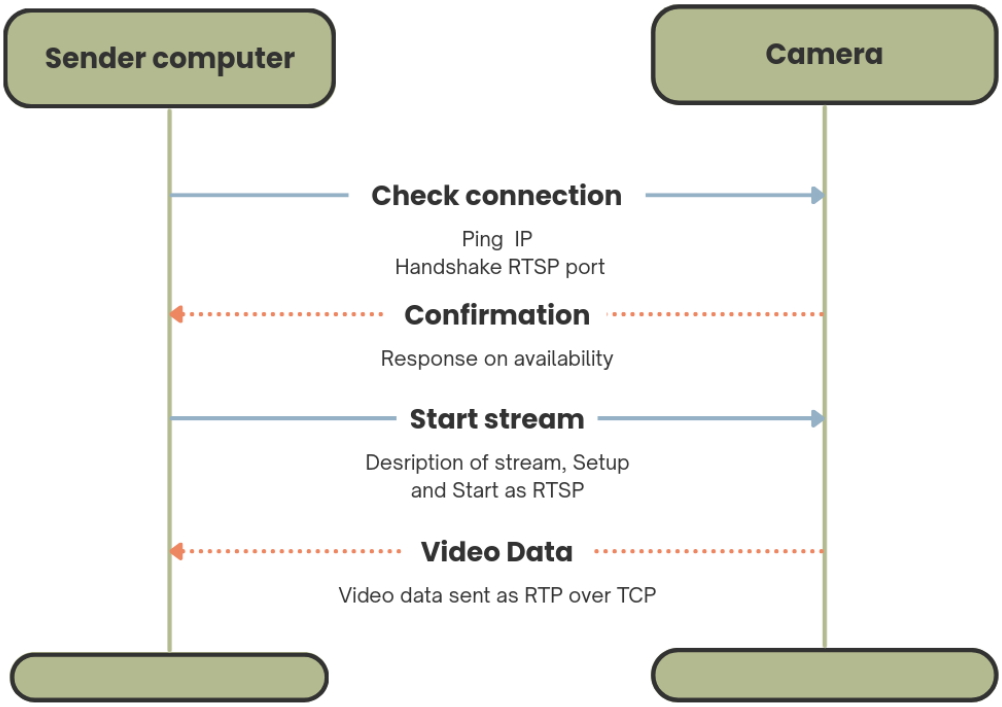

Figure 5: Sequence Diagram between Sender and Camera before active Stream

H.264 video is made up of NAL units (chunks of compromised video). RTP is rules how to split up big NAL units and how to assemble them again. Why Unpack and repack? Control over the stream — you get a clean interception point for your probe config-interval=1 — injecting SPS/PPS every frame so the receiver never gets lost Flexibility — you could insert transcoding, scaling, or other processing between depay and pay if needed later

What is a Probe? In GStreamer, a probe is essentially a spy you attach to a specific point in the pipeline. It lets you intercept data as it flows through, look at it, and optionally modify or drop it — all without interrupting the pipeline itself.

What is a Pad? Before understanding probes you need to understand pads — because probes attach to pads, not to elements directly. Every GStreamer element has pads — they are the connection points between elements:

rtph264pay, Package back into RTP packets, injects SPS/PPS every frame "config-interval-1" udpsink, sends packets to receiver with UDP

Receiver Computer: Listens to UDP port 5000 rtph264depay: Strips RTP again to get H.264 NAL units h264parse: Parses and structures the bitstream avdec-h264: Software decodes H.264 to raw frames (this is the expensive step) autovideosink: Displays on screen (sync=false = no clock sync, lowest latency)

3.3 Pipeline Design

3.4 Timestamps and Latency Measurement Design

3.5 Wi-Fi Network Setup

4 Methodology

4.1 Literature and Knowledge acquisition

The knowledge needed to carry out this thesis was acquired through an iterative process that combined systematic literature review with hands-on practical exploration. This approach, often referred to as an Iterative Literature and Practice-Driven Approach, is characterized by a continuous cycle where theory and practice inform each other rather than following a strict linear sequence [Wohlin et al. (2012)]. Instead of completing all literature review before beginning practical work, the two activities run in parallel, with new practical challenges prompting targeted literature searches and new theoretical insights shaping the direction of experiments. This makes it particularly well suited for applied engineering research where the problem space is not fully known at the outset and understanding develops gradually through doing [Hevner et al. (2004) Saunders et al. (2019)].

The process began by identifying the core knowledge areas required before any development could start. This included understanding the operational context of remote train operation and Automatic Train Operation, the fundamentals of video streaming and relevant transport protocols, and the capabilities and structure of GStreamer as a framework. These areas formed the initial scope of the literature review and gave enough grounding to begin practical work.

Academic literature was gathered primarily through Google Scholar and IEEE Xplore, using search terms related to low-latency video streaming, remote train operation, GStreamer pipeline design, and network performance for real-time applications. Sources were selected based on relevance to the problem, publication quality, and recency where applicable. In addition to academic papers, official technical documentation was used extensively, particularly the GStreamer documentation and protocol specifications for RTP and UDP, which provided the practical detail that academic sources alone could not cover.

Once a sufficient theoretical foundation was in place, practical exploration began through hands-on work with GStreamer. Building and testing pipelines from the command line and through Python quickly revealed challenges that existing literature did not fully address, such as specific configuration trade-offs, unexpected latency behavior, and hardware compatibility issues. Each of these practical obstacles prompted a return to the literature, this time with more precise and targeted questions. New findings were then brought back into the development process, tested, and evaluated against real behaviour.

This cycle of reading, building, observing, and searching again repeated throughout the entire development phase. The result is that the theoretical foundation of this thesis did not precede the practical work, but grew alongside it, shaped by the specific demands of the problem at hand.

4.1.1 Search Strategy and Databases

A multi-platform search approach was utilized to retrieve a wide array of high-quality sources. The primary databases included:

- Scopus: For retrieving peer-reviewed, impactful scientific articles.
- Google Scholar: For broader academic and institutional literature.
- GStreamer docs: For specifics on GStreamer development.

4.1.2 Inclusion Criteria

To ensure the study is based on the relevant and current information, specific criteria were applied to filter the search results. Given the rapid pace of technological change, a focus was placed on

recent publications. Where modern papers have referenced to older papers, I have include some as to say and show where certain numbers are coming from and how they have influenced results.

Table 3: Inclusion Criteria for Literature Review

ID	Inclusion Criteria (IC)
IC1	Publication Date: Between 2020-2026
IC2	Language: Written in English
IC3	Document Type: Primarily "Article", "Conference Paper" or "Thesis"

4.1.3 Search Queries and Refinement

Several digital tools as mentioned below were used for this paper. The main purpose of the usage was to enhance clarity, assisting in latex, and in general increase quality. Specifics include writing reference list in correct format. Structuring sentences and paragraphs. Checking for typos and grammatical errors.

- OpenAI's ChatGPT: Employed as a helpful resource for LaTeX formatting suggestions, generating structured content (tables and lists), translating between Norwegian and English, and reviewing text for synonyms and restructuring ideas to ensure arguments were effectively communicated.
- Google Gemini: Used for similar fields as ChatGPT, but also for general proofreading, and refining sentence structure and tone to maintain a high standard of academic writing.
- Anthropic's Claude.ai for discussion and development of script syntax.
- Visual Studio Code: was used as the main LaTeX editor because of its powerful extensions for LaTeX support, syntax highlighting, and version control integration.

4.2 Validity

4.3 Reliability

4.4 Research Design

This thesis follows an applied research approach, combining Design Science Research (DSR) and experimental research to address both the development and evaluation aspects of the work. This combination is well suited for engineering projects where the goal is to produce a functional artifact while also generating knowledge through systematic testing and measurement.

The DSR component governs the design and development of the video streaming pipeline. Following the principles outlined by Hevner et al. [source], the research contribution is embedded in the artifact itself, meaning the pipeline design, the architectural decisions made, and the reasoning behind them all form part of the research output. The problem is clearly grounded in practice, the solution is built and refined iteratively, and the artifact is evaluated against the requirements defined in the previous chapter.

The experimental research component covers the testing and evaluation phase. Drawing on the framework described by Wohlin et al. [source], the pipeline is tested across a range of configurations in a controlled setting. Variables such as codec choice, payload settings, and network conditions are systematically varied, and end-to-end latency is measured and compared across configurations. This allows conclusions to be drawn about which configurations perform best and what trade-offs exist between latency and video quality.

Together these two approaches reflect the dual nature of the thesis: it is both a development project and a measurement study, and the methodology is designed to support both goals.

4.5 Development

4.5.1 GStreamer Pipeline Development

4.6 Experimental Methodology

4.6.1 Latency Measurement

5 Results

6 Conclusion

6.1 Evaluation of latency

References

[1] Simon Jürgensen. 'Remote Control for Rail Vehicles'. PhD thesis. University of Kiel, 2025. URL: https://rtsys.informatik.uni-kiel.de/~biblio/downloads/theses/sij-bt.pdf (visited on 15th Oct. 2025).

[2] Inzirillo Francesco et al. *D5.4 - Documentation of use cases for Remote Driving*. FP2R2DATO and Europe's Rail, 2023. URL: https://rail-research.europa.eu/wp-content/uploads/2025/07/D5.4-%E2%80%93-Documentation-of-use-cases-for-Remote-Driving.pdf (visited on 8th Dec. 2025).

[3] International Electrotechnical Commission (IEC). *IEC 62290-1:2025 -Railway applications - Urban guided transport management and command/control systems - Part 3: System requirements specification*. IEC, 2025. URL: https://online.standard.no/nb/nek-iec-62290-3-2025.

[4] International Association of Public Transport (UITP). *World Report on Metro Automation*. UITP, 2018. URL: https://www.uitp.org/publications/world-report-on-metro-automation/ (visited on 15th Oct. 2025).

[5] Europe's Rail FP6 WP3. *D3.1 – Use Cases and Scenarios for ATO in Different GoA Demos for G1 Regional Lines*. Tech. rep. Europe's Rail, 2025. URL: https://rail-research.europa.eu/rail-projects/outputs/d3-1-uc-and-scenarios-for-ato-in-different-goa-demos-for-g1-regional-lines/ (visited on 20th Mar. 2026).

[6] Office of Rail and Road. *Railway Safety Principles and Factors for Safe Operation of GoA4 UTO*. Tech. rep. Office of Rail and Road, 2024. URL: https://www.orr.gov.uk/sites/default/files/2024-07/railway-safety-principles-factors-for-safe-operation-of-goa4-uto.pdf (visited on 20th Mar. 2026).

[7] Niels Brandenburger, Friedrich Maximilian Strauß and Anja Naumann. *Effects of Video Quality on Perception and Reaction Time in Video-based Remote Train Control*. DLR, 2023. URL: https://elib.dlr.de/194929/1/PresentationHFES2023_DLR.pdf (visited on 5th Dec. 2025).

[8] Emilia Salvesen Kozarevic. 'Developing Remote Control as a Fallback System: An Iterative Approach'. PhD thesis. NTNU, 2025.

[9] Stefan Neumeier et al. 'Teleoperation: The Holy Grail to Solve Problems of Automated Driving? Sure, but Latency Matters'. In: (2019). URL: https://dl.acm.org/doi/pdf/10.1145/3342197.3344534 (visited on 15th Oct. 2025).

[10] Einar Thingstad Myhrvold. 'Latency for Remote Control in ATO a Scoping Review'. PhD thesis. NTNU, 2025.

[11] ETSI. *ETSI TS 122 289 V17.0.0 LTE 5G Mobile communication system for railways 3GPP TS 22.289 version 17.0.0 Release 17*. Tech. rep. 2022. URL: https://www.etsi.org/deliver/etsi_ts/122200_122299/122289/17.00.00_60/ts_122289v170000p.pdf.

[12] Nacho Celaya Vela et al. *D41.2 - Testing reports and assessment*. CAF et al., 2024. URL: https://rail-research.europa.eu/wp-content/uploads/2025/02/D41.2-Testing-reports-assessment.pdf (visited on 7th Dec. 2025).

[13] Joseph Vannicola et al. 'GPS Time Transfer: Applications to the US Naval Observatory Two-Way Satellite Time and Frequency Transfer'. In: *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control* 59.12 (2012), pp. 2625–2634. URL: https://ieeexplore.ieee.org/document/6378481 (visited on 14th Apr. 2026).

[14] Pratap Misra and Per Enge. *Global Positioning System: Signals, Measurements, and Performance*. 2nd ed. Ganga-Jamuna Press, 2011. URL: https://www.amazon.com/Global-Positioning-System-Measurements-Performance/dp/0970954417.

[15] Ilaria Sesia, Marco Pierozzi and Patrizia Tavella. 'GPS and IEEE 1588 Synchronization for the Measurement of Synchrophasors in Electric Power Systems'. In: *Computer Standards and Interfaces* 33.2 (2010), pp. 176–181. URL: https://www.sciencedirect.com/science/article/abs/pii/S0920548910000875 (visited on 14th Apr. 2026).

[16] Admir Kaknjo et al. 'Real-Time Video Latency Measurement between a Robot and Its Remote Control Station: Causes and Mitigation'. In: *Wireless Communications and Mobile Computing* (2018). URL: https://www.researchgate.net/publication/329369713_Real-Time_Video_Latency_Measurement_between_a_Robot_and_Its_Remote_Control_Station_Causes_and_Mitigation (visited on 15th Oct. 2025).

[17] David Mills et al. *RFC 5905: Network Time Protocol Version 4 – Reference and Implementation Guide*. Tech. rep. IETF, 2010. URL: https://www.rfc-editor.org/rfc/rfc5905 (visited on 18th Mar. 2026).

[18] Chrony Project. *Chrony – Configuration Examples and Accuracy*. 2024. URL: https://chrony-project.org/examples.html (visited on 2nd Apr. 2026).

[19] Red Hat. *Configuring Time Synchronization – Red Hat Enterprise Linux 9*. 2024. URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_basic_system_settings/configuring-time-synchronization_configuring-basic-system-settings (visited on 25th Mar. 2026).

[20] Red Hat. *Chrony with Hardware Timestamping – Red Hat Enterprise Linux 10*. 2024. URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_time_synchronization/chrony-with-hw-timestamping (visited on 25th Mar. 2026).

[21] Iraj Sodagar. 'The MPEG-DASH Standard for Multimedia Streaming Over the Internet'. In: *IEEE MultiMedia* 18.4 (2011), pp. 62–67. URL: https://ieeexplore.ieee.org/document/6077864 (visited on 15th Feb. 2026).

[22] Daniel Mejías et al. *Towards Railways Remote Driving: Analysis of Video Streaming Latency and Adaptive Rate Control*. 2024. URL: https://ieeexplore.ieee.org/document/10597107 (visited on 6th Dec. 2025).

[23] Timo Blomqvist. 'Video latency in a vehicle's remoteoperating system'. PhD thesis. 2018. URL: https://www.theseus.fi/bitstream/handle/10024/157223/blomqvist_timo.pdf?sequence=1&isAllowed=y.

[24] Iain E. Richardson. *The H.264 Advanced Video Compression Standard*. 2nd ed. Wiley, 2010. URL: https://last.hit.bme.hu/download/vidtech/k%C3%B6nyvek/Iain%20E.%20Richardson%20-%20H264%20(2nd%20edition).pdf.

[25] Thomas Wiegand et al. 'Overview of the H.264/AVC Video Coding Standard'. In: *IEEE Transactions on Circuits and Systems for Video Technology* 13.7 (2003), pp. 560–576. URL: https://ieeexplore.ieee.org/document/1218189 (visited on 10th Feb. 2026).

[26] Gary J. Sullivan, Pankaj Topiwala and Ajay Luthra. 'The H.264/AVC Advanced Video Coding Standard: Overview and Introduction to the Fidelity Range Extensions'. In: *Proceedings of SPIE* 5558 (2004). URL: https://www.ijcaonline.org/archives/volume181/number14/aljammas-2018-ijca-917575.pdf (visited on 20th May 2026).

[27] Gary J. Sullivan et al. 'Overview of the High Efficiency Video Coding (HEVC) Standard'. In: *IEEE Transactions on Circuits and Systems for Video Technology* 22.12 (2012), pp. 1649–1668. URL: https://ieeexplore.ieee.org/document/6316136 (visited on 10th Feb. 2026).

[28] Jens-Rainer Ohm et al. 'Comparison of the Coding Efficiency of Video Coding Standards Including High Efficiency Video Coding (HEVC)'. In: *IEEE Transactions on Circuits and Systems for Video Technology* 22.12 (2012), pp. 1669–1684. URL: https://ieeexplore.ieee.org/document/6317158 (visited on 10th Feb. 2026).

[29] Patrick Seeling and Martin Reisslein. 'Video Traffic Characteristics of Modern Encoding Standards: H.264/AVC with SVC and MVC Extensions and H.265/HEVC'. In: *The Scientific World Journal* (2014). URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC3951063/ (visited on 13th Feb. 2026).

[30] VOCAL Technologies. *What are H.264 NAL Units?* 2023. URL: https://vocal.com/video-codecs/itu-h-264/what-are-h-264-nal-units/ (visited on 13th Feb. 2026).

[31] Ye-Kui Wang et al. *RFC 6184: RTP Payload Format for H.264 Video*. Tech. rep. IETF, 2011. URL: https://www.rfc-editor.org/rfc/rfc6184 (visited on 13th Feb. 2026).

[32] Wikipedia contributors. *Motion JPEG*. 2026. URL: https://en.wikipedia.org/wiki/Motion_JPEG (visited on 15th Feb. 2026).

[33] e-con Systems. *A Comprehensive Study: MJPEG and H.264 Compression in Embedded Vision*. 2024. URL: https://www.e-consystems.com/blog/camera/technology/a-comprehensive-study-mjpeg-and-h-264-compression-in-embedded-vision/ (visited on 15th Feb. 2026).

[34] Admir Kaknjo et al. 'Real-Time Video Latency Measurement between a Robot and Its Remote Control Station: Causes and Mitigation'. In: *Wireless Communications and Mobile Computing* (2018). URL: https://www.researchgate.net/publication/329369713_Real-Time_Video_Latency_Measurement_between_a_Robot_and_Its_Remote_Control_Station_Causes_and_Mitigation (visited on 15th Oct. 2025).

[35] Henning Schulzrinne et al. *RFC 3550: RTP: A Transport Protocol for Real-Time Applications*. Tech. rep. IETF, 2003. URL: https://www.rfc-editor.org/rfc/rfc3550 (visited on 10th Feb. 2026).

[36] Mark Baugher et al. *RFC 3711: The Secure Real-time Transport Protocol (SRTP)*. Tech. rep. IETF, 2004. URL: https://www.rfc-editor.org/rfc/rfc3711 (visited on 5th Apr. 2026).

[37] Jon Postel. *RFC 768: User Datagram Protocol*. Tech. rep. IETF, 1980. URL: https://datatracker.ietf.org/doc/html/rfc768 (visited on 17th Mar. 2026).

[38] Behrouz A. Forouzan. *Data Communications and Networking*. 5th ed. McGraw-Hill, 2012. URL: https://jcer.in/jcer-docs/E-Learning/Digital%20Library%20/E-Books/Data-Communications-and-Network-5e.pdf (visited on 17th Mar. 2026).

[39] Jon Postel. *RFC 793: Transmission Control Protocol*. Tech. rep. IETF, 1981. URL: https://www.rfc-editor.org/rfc/rfc793 (visited on 20th May 2026).

[40] Margaret H. Pinson and Stephen Wolf. 'The Relationship Among Video Quality, Screen Resolution, and Bit Rate'. In: *IEEE Transactions on Broadcasting* 58.2 (2012), pp. 258–262. URL: https://ieeexplore.ieee.org/document/5729848 (visited on 14th May 2026).

[41] Florin Dobrian et al. 'Understanding the Impact of Video Quality on User Engagement'. In: *ACM SIGCOMM*. 2011. URL: https://dl.acm.org/doi/10.1145/2018436.2018478 (visited on 20th May 2026).

[42] Axis Communications. *Latency in Live Network Video Surveillance*. 2024. URL: https://www.axis.com/dam/public/9d/e4/5d/latency-in-live-network-video-surveillance-en-US-190945.pdf (visited on 20th May 2026).

[43] Wassim Hamidouche et al. 'Image and Video Coding Techniques for Ultra-low Latency'. In: *ACM Computing Surveys* 55.3 (2022). URL: https://dl.acm.org/doi/10.1145/3512342 (visited on 20th May 2026).

[44] Pierre Leray et al. 'Ultrahigh temporal resolution of visual presentation using gaming monitors and G-Sync'. In: *Behavior Research Methods* 50 (2018), pp. 26–38. URL: https://link.springer.com/article/10.3758/s13428-017-1003-6 (visited on 20th May 2026).

[45] GStreamer Project. *What is GStreamer?* 2024. URL: https://gstreamer.freedesktop.org/documentation/application-development/introduction/gstreamer.html (visited on 20th May 2026).

[46] GStreamer Project. *Elements*. 2024. URL: https://gstreamer.freedesktop.org/documentation/application-development/basics/elements.html (visited on 20th May 2026).

[47] GStreamer Project. *Pads and Capabilities*. 2024. URL: https://gstreamer.freedesktop.org/documentation/application-development/basics/pads.html (visited on 20th May 2026).

[48] GStreamer Project. *Sink Elements*. 2024. URL: https://gstreamer.freedesktop.org/documentation/additional/design/element-sink.html (visited on 20th May 2026).

[49] GStreamer Project. *Probes*. 2024. URL: https://gstreamer.freedesktop.org/documentation/additional/design/probes.html (visited on 20th May 2026).

[50] GStreamer Project. *Pipeline Manipulation*. 2024. URL: https://gstreamer.freedesktop.org/documentation/application-development/advanced/pipeline-manipulation.html (visited on 20th May 2026).

[51] IEEE. *IEEE 802.11-2020: IEEE Standard for Information Technology – Telecommunications and Information Exchange Between Systems – Local and Metropolitan Area Networks – Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*. 2020. URL: https://standards.ieee.org/ieee/802.11/7028/ (visited on 20th May 2026).

[52] Dmitry Bankov et al. 'Enabling Real-Time Applications in Wi-Fi Networks'. In: *International Journal of Distributed Sensor Networks* (2019). URL: https://journals.sagepub.com/doi/full/10.1177/1550147719845312 (visited on 20th May 2026).

[53] Zexian Li et al. '5G URLLC: Design Challenges and System Concepts'. In: *IEEE 15th International Symposium on Wireless Communication Systems* (2018). URL: https://ieeexplore.ieee.org/document/8491078 (visited on 20th May 2026).

[54] Philipp Schulz et al. 'Latency Critical IoT Applications in 5G: Perspective on the Design of Radio Interface and Network Architecture'. In: *IEEE Communications Magazine* 55.2 (2017), pp. 70–78. URL: https://ieeexplore.ieee.org/document/7842391 (visited on 20th May 2026).

[55] J. Ouden, V. Ho and T. Smagt. 'Design and Evaluation of Remote Driving Architecture on 4G and 5G Mobile Networks'. In: (2022). URL: https://www.frontiersin.org/journals/future-transportation/articles/10.3389/ffutr.2021.801567/full (visited on 5th Dec. 2025).

[56] Tailscale Inc. *What is Tailscale?* 2024. URL: https://tailscale.com/kb/1151/what-is-tailscale (visited on 20th May 2026).

[57] Jason A. Donenfeld. 'WireGuard: Next Generation Kernel Network Tunnel'. In: *Network and Distributed System Security Symposium (NDSS)*. 2017. URL: https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/wireguard-next-generation-kernel-network-tunnel/ (visited on 20th May 2026).

[58] Prasad Calyam et al. 'A Loss and Recovery Model for RTP Media Streams'. In: *IEEE International Conference on Communications*. 2012. URL: https://ieeexplore.ieee.org/document/6364439 (visited on 20th May 2026).

[59] Slobodan Pekec, Dragan Vucic and Vladimir Blagojevic. 'Impact of Packet Loss Rate on Quality of Compressed High Resolution Videos'. In: *Sensors* (2023). URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC10007065/ (visited on 20th May 2026).

[60] Linode. *How to Use the System Activity Reporter (sar)*. 2021. URL: https://www.linode.com/docs/guides/how-to-use-sar/ (visited on 20th May 2026).

[61] Umme Sara, Morium Akter and Mohammad Shorif Uddin. 'Image Quality Assessment through FSIM, SSIM, MSE and PSNR: A Comparative Study'. In: *Journal of Computer and Communications* 7.3 (2019), pp. 8–18. URL: https://www.scirp.org/journal/paperinformation?paperid=90911 (visited on 20th May 2026).

[62] Anish Mittal, Anush Krishna Moorthy and Alan Conrad Bovik. 'No-Reference Image Quality Assessment in the Spatial Domain'. In: *IEEE Transactions on Image Processing* 21.12 (2012), pp. 4695–4708. URL: https://live.ece.utexas.edu/publications/2012/TIP%20BRISQUE.pdf (visited on 20th May 2026).